

Documento de Proposta

Projeto Programação 4

Guilherme Machado Campos Gustavo Rigon do Prado Vinícius Nunes de Oliveira

21. Junho 2026

1 Proposta

1.1 Tema

Um site especializado em ensinar novos donos como adestrar seu animal de estimação.

1.2 Justificativa

Tendo em vista que as pessoas têm dificuldade em treinar seus animais de estimação e que tutoriais nem sempre seguem uma ordem correta de progressão, decidimos criar um site que resolva esse problema.

Por isso, queremos criar um local que agregue técnicas de adestramento com tutoriais de como fazer brinquedos para enriquecimento ambiental.

1.3 Público alvo

Todos que possuem animais ou pretendem possuir.

1.4 Objetivos do Sistema

- Englobar um conjunto de tutoriais num lugar só.
- Criar tutoriais de adestramento com progressão.
- Criar tutoriais de brinquedos de enriquecimento ambiental.

1.5 Repositório

O repositório está disponível publicamente no endereço <https://github.com/viccios/projeto-prog4>.

2 Requisitos Funcionais e Não funcionais

Os requisitos funcionais e não funcionais estão disponíveis no endereço <https://github.com/viccios/projeto-prog4/blob/main/docs/requirements.md>.

3 Diagramas de caso de uso e Histórias do Usuário

O diagrama de caso de uso e as histórias do usuário estão disponíveis no endereço <https://github.com/viccios/projeto-prog4/blob/main/docs/use-case-diagram.md>.

4 Diagrama Entidade-Relacionamento (DER)

O diagrama entidade-relacionamento está disponível no endereço <https://github.com/viccios/projeto-prog4/blob/main/docs/erd.md>.

5 Script de Criação de Tabelas (DDL)

O script de criação de tabelas (DDL) está disponível no endereço <https://github.com/viccios/projeto-prog4/blob/main/backend/src/database/schema.sql>. O SGBD escolhido para o projeto é o MySQL.

O script inicial está disponível abaixo:

```
CREATE DATABASE IF NOT EXISTS animal_training
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE animal_training;

CREATE TABLE users (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  user_name VARCHAR(50) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL
);

CREATE TABLE pets (
  pet_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  pet_name VARCHAR(50) NOT NULL,
  animal_species ENUM('dog', 'cat') NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

CREATE TABLE courses (
  course_id INT PRIMARY KEY AUTO_INCREMENT,
  course_name VARCHAR(50) NOT NULL,
  description TEXT NOT NULL,
  animal_species ENUM('dog', 'cat') NOT NULL,
  importance ENUM('normal', 'important', 'very_important') NOT NULL
);

CREATE TABLE lessons (
```

```
    lesson_id INT PRIMARY KEY AUTO_INCREMENT,  
    lesson_name VARCHAR(50) NOT NULL,  
    description TEXT NOT NULL,  
    youtube_video_url VARCHAR(500) NOT NULL,  
    requires_course_access BOOLEAN NOT NULL DEFAULT FALSE  
);  
  
CREATE TABLE tags (  
    tag_id INT PRIMARY KEY AUTO_INCREMENT,  
    tag_name VARCHAR(25) NOT NULL UNIQUE  
);  
  
CREATE TABLE course_lesson (  
    course_id INT NOT NULL,  
    lesson_id INT NOT NULL,  
    order_number INT NOT NULL CHECK (order_number > 0),  
    UNIQUE (course_id, order_number),  
    PRIMARY KEY (course_id, lesson_id),  
    FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE  
CASCADE,  
    FOREIGN KEY (lesson_id) REFERENCES lessons(lesson_id) ON DELETE  
CASCADE  
);  
  
CREATE TABLE course_tag (  
    course_id INT NOT NULL,  
    tag_id INT NOT NULL,  
    PRIMARY KEY (course_id, tag_id),  
    FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE  
CASCADE,  
    FOREIGN KEY (tag_id) REFERENCES tags(tag_id) ON DELETE CASCADE  
);  
  
CREATE TABLE pet_course_progress (  
    course_id INT NOT NULL,  
    pet_id INT NOT NULL,  
    grade INT NOT NULL DEFAULT 0 CHECK (grade BETWEEN 0 AND 10),  
    is_completed BOOLEAN NOT NULL DEFAULT FALSE,  
    PRIMARY KEY (course_id, pet_id),  
    FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE  
CASCADE,  
    FOREIGN KEY (pet_id) REFERENCES pets(pet_id) ON DELETE CASCADE  
);  
  
CREATE TABLE pet_lesson_progress (  
    lesson_id INT NOT NULL,  
    pet_id INT NOT NULL,  
    grade INT NOT NULL DEFAULT 0 CHECK (grade BETWEEN 0 AND 10),  
    is_completed BOOLEAN NOT NULL DEFAULT FALSE,  
    PRIMARY KEY (lesson_id, pet_id),  
    FOREIGN KEY (lesson_id) REFERENCES lessons(lesson_id) ON DELETE  
CASCADE,  
    FOREIGN KEY (pet_id) REFERENCES pets(pet_id) ON DELETE CASCADE  
);
```

O esquema foi testado com o SGBD MariaDB (compatível com MySQL) rodando em um sistema Linux.

```
$ mariadb --version
mariadb from 11.4.9-MariaDB, client 15.2 for Linux (x86_64) using
readline 5.1
```

O esquema criado é experimental e pode sofrer várias mudanças ao longo do tempo, servindo atualmente como um exemplo de como o banco será modelado no futuro.

6 Estrutura de Pastas do Projeto (MVC)

O projeto está estruturado por meio de um monorepo (repositório único que contém tanto o frontend quanto o backend da aplicação), estruturado utilizando a arquitetura MVC (*Model, View, Controller*). A arquitetura do projeto tem como foco o desacoplamento de código, aumentando sua coesão e reusabilidade, facilitando a manutenção.

Exemplo de estrutura de pastas:

```
.
├── backend
│   ├── eslint.config.js
│   ├── package.json
│   ├── package-lock.json
│   └── src
│       ├── app.js
│       ├── app.test.js
│       ├── controllers
│       ├── models
│       ├── routes
│       │   └── hello-world.routes.js
│       ├── server.js
│       └── services
├── docs
│   ├── architecture.md
│   ├── erd.md
│   ├── proposal-document
│   │   ├── proposal-document.pdf
│   │   └── proposal-document.typ
│   ├── README.md
│   └── requirements.md
├── frontend
└── README.md
```

6.1 Principais características:

- Configuração do Eslint e Prettier para *linting* e formatação do código.
- Uso do *framework* de testes Vitest para a criação de testes unitários.
- Uso da arquitetura MVC para a modularização do projeto.
- Documentação através de arquivos `.md` e documento de proposta em Typst.
- Documentação gerada automaticamente via OpenAPI (anteriormente Swagger) e Scalar UI.
- Deploy automático usando a plataforma Render.

Além disso, o projeto segue boas práticas, como o uso dos *middlewares* `helment`, `morgan` e `rate-limit` para garantir a segurança e estabilidade da *API*.

7 Arquivo `.env.example` e `.gitignore`

Os arquivos `.env.example` e `.gitignore` estão disponíveis no repositório do projeto nos endereços <https://github.com/viccios/projeto-prog4/blob/main/backend/.env.example> e <https://github.com/viccios/projeto-prog4/blob/main/backend/.gitignore>, respectivamente.

O arquivo `.gitignore` foi baixado do repositório <https://github.com/viccios/projeto-prog4/blob/main/backend/.gitignore>, que fornece arquivos `.gitignore` com padrões sensatos (*sensible defaults*) para diversas linguagens.

8 README com a Descrição do Projeto

O arquivo `README.md` com a descrição do projeto, incluindo instruções de instalação e configuração está disponível em <https://github.com/viccios/projeto-prog4/blob/main/README.md>.